



Agente Conversacional para Admisión y Nivelación de la UG

Proyecto II

Facultad de Ciencias Matemáticas y Físicas

Carrera de Ciencia de Datos e Inteligencia Artificial

Procesamiento de Lenguaje Natural

Integrantes:

Alvarado Reyes Verónica Nathaly & John David Ortega Echeverria

Docente: Ing. Angel Cuenca Ortega

Fecha: 10 de julio de 2026

Introducción al problema abordado

En cada período académico, los procesos de admisión y nivelación de la Universidad de Guayaquil (UG) es cuando se generan un volumen alto de consultas repetitivas por parte de los aspirantes y estudiantes como sería en: requisitos de inscripción, fechas de registro, cronograma según el documento de identidad, matrícula, homologación, becas y otros trámites administrativos que tengan los aspirantes. Gran parte de esta información se encuentra dispersa entre el portal oficial, redes sociales y guías externas, lo que dificulta una consulta ágil.

Para comprender esta necesidad se desarrolló un agente conversacional (chatbot) que tiene base en Procesamiento de Lenguaje Natural (PLN), que responde en español preguntas frecuentes sobre admisión, nivelación y vida universitaria a partir de una base de conocimiento propia. En este proyecto se va a centrar en la detección de intenciones mediante similitud textual y la recuperación de respuestas predefinidas, sin recurrir a modelos generativos ni a técnicas de aprendizaje profundo, conforme al alcance definido para esta actividad.

Diseño de intenciones y entidades

La base de conocimiento (`data/base_conocimiento.json`) contiene 87 registros distribuidos en 12 intenciones o temas principales: `carrera`, `admission`, `nivelacion`, `cronograma` (por `cédula/pasaporte`), `matricula_ordinaria` (incluye estudiantes de niveles regulares como cuarto semestre), `tercera_matricula`, `retiro_asignaturas`, `homologacion`, `becas_ayudas`, `carnet_estudiantil`, `biblioteca` y `contacto_soporte`. A esto se suman dos intenciones adicionales — `saludo` y `despedida` — resueltas mediante reglas, sin pasar por el cálculo de similitud.

Cada registro incluye los campos `tipo`, `titulo`, `descripcion` y `detalle`, que se concatenan y preprocesan para construir el vocabulario de comparación. Las intenciones con mayor variabilidad de expresión (por ejemplo, `homologación` o `nivelación`) cuentan con más de un registro, cada uno redactado con vocabulario distinto, para ampliar la cobertura léxica del modelo TF-IDF.

En cuanto a las entidades, se identifican mediante expresiones regulares: (1) el último dígito de la `cédula` o `pasaporte`, usado para calcular el día de postulación según el cronograma oficial; y (2) patrones de `saludo` ("hola", "buenas tardes", "qué tal", entre otros) y de `despedida` o `agradecimiento` ("gracias", "adiós", "hasta luego"), que activan una respuesta fija independiente del motor de similitud.

Descripción de la arquitectura del agente

El proyecto está organizado en tres capas:

- Datos (`data/`): `base_conocimiento.json/.csv` con las intenciones y respuestas, y `consultas_prueba.csv` con el set de evaluación.
- Lógica (`src/`): `preprocessing.py` (normalización, stopwords y stemming), `entities.py` (reglas de `cédula`, `saludo` y `despedida`) y `chatbot.py` (clase `ChatbotUG`, que integra el vectorizador TF-IDF, la similitud coseno y las reglas).
- Interfaz (`app_consola.py` y `notebooks/Chatbot_UG_Colab_Gradio.ipynb`): ejecución local por consola y una interfaz web equivalente construida con Gradio para Google Colab.

El flujo de respuesta sigue este orden: (1) si la consulta está vacía, se solicita reformularla; (2) si coincide con un patrón de `saludo` o `despedida`, se responde de forma fija; (3) si contiene un dígito de `cédula/pasaporte` junto a palabras relacionadas con inscripción, se calcula el día de postulación; (4) en cualquier otro caso, la consulta se preprocesa y se compara mediante similitud coseno contra la matriz TF-IDF de la base de conocimiento, devolviendo la respuesta de mayor similitud si supera el umbral, o un

mensaje de fallback en caso contrario. Todo el proceso está envuelto en un bloque try/except para evitar que una entrada inesperada detenga la ejecución.

Justificación de las decisiones de diseño

TF-IDF frente a una bolsa de palabras con conteos simples

Se utilizó TF-IDF (Term Frequency – Inverse Document Frequency) en lugar de un conteo simple de palabras porque el factor IDF penaliza los términos que aparecen en casi todos los registros (por ejemplo, "universidad" o "estudiante") y da mayor peso a los términos distintivos de cada intención (por ejemplo, "homologación" o "carné"). Con un conteo simple, esas palabras muy frecuentes dominarían la representación vectorial y dificultarían diferenciar entre intenciones; con TF-IDF, la similitud coseno se concentra en el vocabulario que realmente distingue un tema de otro.

Similitud coseno frente a distancia euclidiana

Se eligió la similitud coseno porque mide el ángulo entre los vectores de la consulta y de cada registro, sin verse afectada por su longitud. La distancia euclidiana, en cambio, es sensible a la magnitud de los vectores: una consulta corta y un registro largo pueden compartir exactamente el mismo vocabulario relevante y, aun así, obtener una distancia euclidiana alta simplemente porque uno tiene más palabras que el otro. La similitud coseno normaliza este efecto y permite comparar de forma justa consultas breves ("¿modalidad de Medicina?") contra registros más extensos.

Preprocesamiento: eliminación de stopwords y stemming

Antes de vectorizar, cada texto pasa por normalización (minúsculas, eliminación de tildes y signos), eliminación de stopwords (lista propia en español, sin depender de descargas externas) y stemming con SnowballStemmer de NLTK. Esto reduce palabras con la misma raíz a una forma común: "homologación" y "homologar" se reducen ambas a "homolog", y "aprobación" y "aprobar" a "aprob". Gracias a esto, una consulta como "¿cómo homologar materias?" coincide correctamente con el registro titulado "Homologación de materias", aunque el usuario no haya escrito la palabra exacta que aparece en la base de conocimiento.

Umbral de confianza y respuesta de fallback

El umbral se fijó en 0.18 de forma empírica: se probó el agente con el set de consultas de prueba y se ajustó el valor hasta encontrar un punto en el que las consultas relacionadas con la base de conocimiento superaran el umbral, mientras que las consultas sin relación (o con errores tipográficos severos) quedaran por debajo. Sin este umbral, el sistema siempre devolvería el registro de mayor similitud aunque esta fuera prácticamente nula, entregando respuestas irrelevantes en lugar de admitir que no entendió la consulta. El mensaje de fallback, además, orienta explícitamente al usuario sobre qué temas puede preguntar.

Separación de los datos (JSON) respecto del código

Las intenciones, respuestas y metadatos se almacenan en un archivo JSON independiente del código fuente. Esta separación permite actualizar, corregir o ampliar la base de conocimiento (por ejemplo, agregar una nueva intención sobre becas) sin modificar ni volver a probar la lógica del agente. También facilita que distintas interfaces (consola y Gradio) reutilicen exactamente la misma fuente de datos, evitando duplicación e inconsistencias entre ambas versiones.

Manejo de errores con try/except

El método responder() de la clase ChatbotUG envuelve el flujo de reglas y similitud en un bloque try/except. Si ocurre un error inesperado —por ejemplo, un carácter no soportado o un problema al vectorizar una entrada anómala— el agente captura la excepción y devuelve un mensaje explicativo en lugar de detener la ejecución. Antes de esto, además, se valida que la consulta no esté vacía o compuesta solo de espacios, devolviendo una instrucción clara para el usuario.

Resultados de la evaluación

La evaluación se realizó con el script evaluar.py sobre un conjunto de 25 consultas variadas (data/consultas_prueba.csv), que incluye preguntas bien formuladas, variaciones lingüísticas, errores tipográficos deliberados y una consulta sin relación con el dominio. Los resultados globales fueron:

- **Accuracy: 92 % (23/25 consultas clasificadas correctamente)**
- **F1-macro: 0.94**

Intención	Precisión	Recall	F1-score
admission	1.00	1.00	1.00
becas_ayudas	1.00	1.00	1.00
biblioteca	1.00	1.00	1.00
carnet_estudiantil	1.00	1.00	1.00
carrera	1.00	0.67	0.80
contacto_soporte	1.00	1.00	1.00
cronograma_cedula	1.00	1.00	1.00
despedida	1.00	1.00	1.00
fallback	0.33	1.00	0.50
homologacion	1.00	1.00	1.00
matricula_ordinaria	1.00	1.00	1.00
nivelacion	1.00	0.75	0.86
retiro_asignaturas	1.00	1.00	1.00
saludo	1.00	1.00	1.00
tercera_matricula	1.00	1.00	1.00

Los dos errores registrados corresponden a consultas con errores tipográficos severos (por ejemplo, "komo aprwebo nivelasion" y "kiero saber sobre medicina"), en las que varias palabras clave quedaron tan alteradas que, tras el stemming, no comparten raíz alguna con los términos del vocabulario. En ambos casos el sistema devolvió un fallback en lugar de una intención incorrecta, lo cual es un comportamiento seguro: el agente prefiere admitir que no entendió antes que ofrecer información equivocada.

Limitaciones observadas

El enfoque basado en TF-IDF y similitud coseno no incorpora corrección ortográfica ni comprensión semántica de sinónimos no presentes en el vocabulario de entrenamiento; por lo tanto, errores tipográficos severos o formulaciones muy indirectas pueden no ser reconocidos. Asimismo, algunas intenciones cuentan con pocos registros de ejemplo, lo que las hace más sensibles a la forma exacta en que el usuario formula la pregunta. Una extensión futura —fuera del alcance definido para esta actividad— podría incorporar corrección ortográfica básica o embeddings semánticos preentrenados para mitigar estas limitaciones.

Conclusiones

El agente conversacional desarrollado cumple el objetivo de identificar la intención del usuario y devolver respuestas pertinentes sobre admisión, nivelación y trámites académicos de la Universidad de Guayaquil, utilizando exclusivamente técnicas clásicas de PLN: normalización, eliminación de stopwords, stemming, representación TF-IDF y similitud coseno. La separación entre datos y lógica, el manejo de reglas para saludo/despedia y cronograma por cédula, y el mecanismo de fallback contribuyen a que el sistema sea robusto ante entradas inesperadas. La evaluación cuantitativa (92 % de accuracy, F1-macro de 0.94) confirma que el enfoque es efectivo para el dominio acotado del proyecto, aunque persisten limitaciones esperables frente a errores tipográficos severos, propias de un modelo sin corrección ortográfica ni embeddings semánticos.